

National University of Computer and Emerging Sciences



DL-2001: Introduction to Data Science Lab Manual 13

Department of Data Science
FAST-NU, Lahore, Pakistan

Table of Contents

Objectives	3
1 Density Based Clustering.....	3
2 DBSCAN.....	3
2.1 Key Parameters in DBSCAN	4
2.2 How does DBSCAN work?	4
2.3 Steps in the DBSCAN Algorithm.....	5
2.4 Steps in the DBSCAN Algorithm.....	5
2.5 Python Implementation using Built in Library.....	6
2.6 From scratch	10
2.7 When Should We Use DBSCAN Over K-Means Clustering?	10

Objectives

After performing this lab, students shall be able to:

- ✓ Understand DBSCAN from scratch
- ✓ Perform DBSCAN clustering using built in libraries

1 Density Based Clustering

Density-based clustering is a method that groups points that are closely packed together, while marking points in sparse regions as noise or outliers.

It identifies clusters by looking for areas of high point density, allowing it to discover arbitrarily shaped clusters and handle noise well.

DBSCAN is the most common algorithm in this category.

2 DBSCAN

DBSCAN is a density-based clustering algorithm that groups data points that are closely packed together and marks outliers as noise based on their density in the feature space. It identifies clusters as dense regions in the data space separated by areas of lower density. Unlike K-Means or hierarchical clustering which assumes clusters are compact and spherical, DBSCAN perform well in handling real-world data irregularities such as:

- Arbitrary-Shaped Clusters: Clusters can take any shape not just circular or convex.
- Noise and Outliers: It effectively identifies and handles noise points without assigning them to any cluster.



The figure above shows a data set with clustering algorithms: K-Means and Hierarchical handling compact, spherical clusters with varying noise tolerance while DBSCAN manages arbitrary-shaped clusters and noise handling.

2.1 Key Parameters in DBSCAN

Unsupervised Learning Algorithms allow users to perform more complex processing tasks compared to supervised learning. Although, unsupervised learning can be more unpredictable compared with other natural learning methods. Unsupervised learning algorithms include clustering, anomaly detection, neural networks, etc.

2.1.1 eps

This defines the radius of the neighborhood around a data point. If the distance between two points is less than or equal to eps they are considered neighbors. A common method to determine eps is by analyzing the k-distance graph. Choosing the right eps is important:

- If eps is too small most points will be classified as noise.
- If eps is too large clusters may merge and the algorithm may fail to distinguish between them.

2.1.1 MinPts

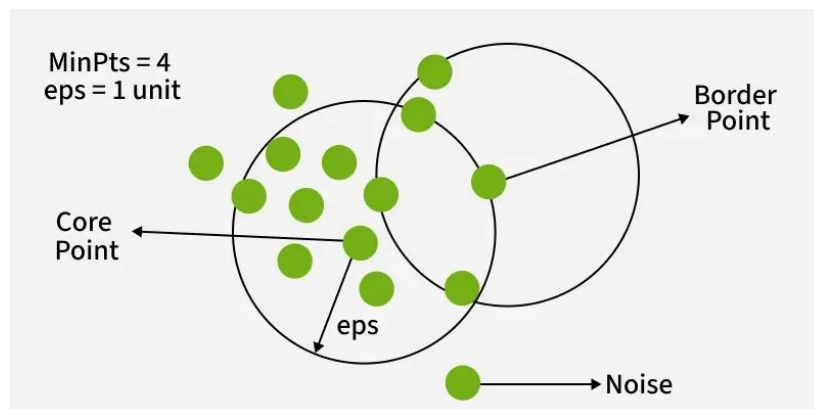
This is the minimum number of points required within the eps radius to form a dense region. A general rule of thumb is to set $\text{MinPts} \geq D+1$ where D is the number of dimensions in the dataset.

2.2 How does DBSCAN work?

DBSCAN works by categorizing data points into three types:

1. **Core points** which have a sufficient number of neighbors within a specified radius (epsilon)
2. **Border points** which are near core points but lack enough neighbors to be core points themselves
3. **Noise points** which do not belong to any cluster.

By iteratively expanding clusters from core points and connecting density-reachable points, DBSCAN forms clusters without relying on rigid assumptions about their shape or size.



2.3 Steps in the DBSCAN Algorithm

1. **Identify Core Points:** For each point in the dataset count the number of points within its eps neighborhood. If the count meets or exceeds MinPts mark the point as a core point.
2. **Form Clusters:** For each core point that is not already assigned to a cluster create a new cluster. Recursively find all density-connected points i.e points within the eps radius of the core point and add them to the cluster.
3. **Density Connectivity:** Two points a and b are density-connected if there exists a chain of points where each point is within the eps radius of the next and at least one point in the chain is a core point. This chaining process ensures that all points in a cluster are connected through a series of dense regions.
4. **Label Noise Points:** After processing all points any point that does not belong to a cluster is labeled as noise.

2.4 Steps in the DBSCAN Algorithm

To assess the quality and effectiveness of clustering algorithms, quantitative evaluation metrics are essential. These metrics help determine how well the algorithm has separated the data into meaningful groups and how closely the resulting clusters match the underlying structure of the dataset. In DBSCAN, we adopt two widely used measures, Silhouette Score and Adjusted Rand Score which together provide both internal and external perspectives on clustering performance.

2.4.1 Silhouette Score

The Silhouette Score is an internal evaluation metric that measures the cohesion and separation of clusters without requiring ground-truth labels. It quantifies how similar each data point is to its assigned cluster compared with other clusters. The score ranges from -1 to 1 :

- Values near $+1$ indicate well-defined clusters in which points are tightly grouped within their own cluster and clearly separated from others.
- Values near 0 suggest that clusters overlap or that points lie close to cluster boundaries.
- Values near -1 imply poor assignment, where points are closer to neighboring clusters than to their own.

A higher overall Silhouette Score signifies better clustering structure and more meaningful partitions.

For more information: <https://www.geeksforgeeks.org/machine-learning/what-is-silhouette-score/>

2.4.2 Adjusted Rand Score (Adjusted Rand Index)

The Adjusted Rand Score (ARI) is an external evaluation metric that compares the similarity between the predicted cluster assignments and the true class labels, while adjusting for agreement that could occur by chance. The score ranges from 0 to 1:

- Scores above 0.9 represent excellent cluster recovery, indicating a near-perfect alignment with the ground truth.
- Scores above 0.8 denote good recovery and strong consistency with the true labels.
- Scores below 0.5 reflect weak recovery, suggesting that the clustering structure diverges significantly from the true classes.

Because ARI corrects for random label matching, it provides a robust measurement of clustering accuracy, even when the number or size of clusters varies.

For more information: <https://www.geeksforgeeks.org/machine-learning/rand-index-in-machine-learning/>

2.5 Python Implementation using Built in Library

2.5.1 Import Libraries

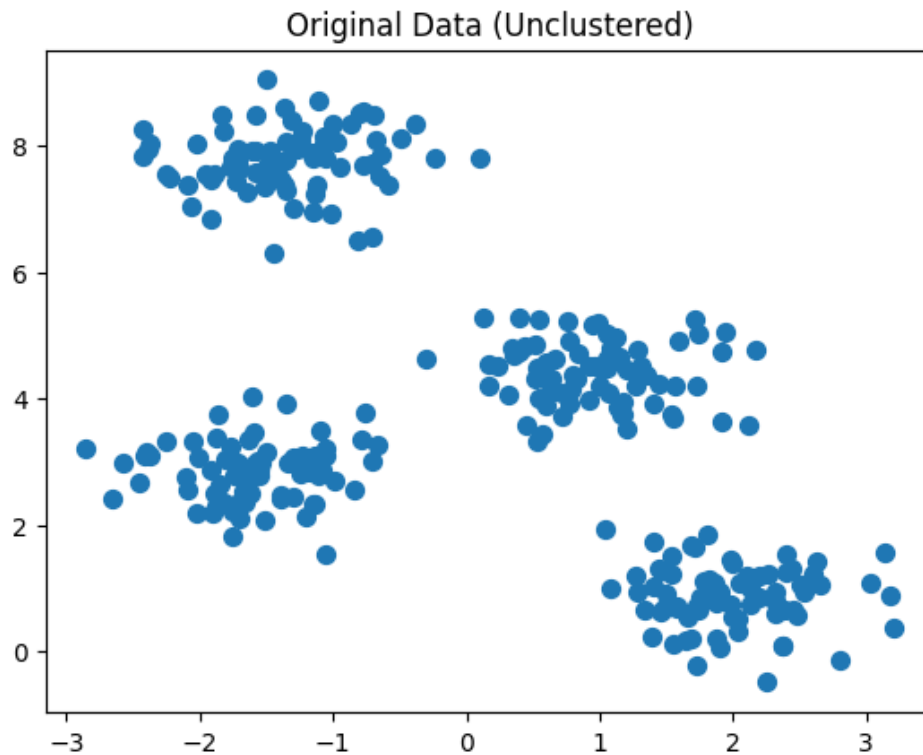
```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.cluster import DBSCAN
from sklearn import metrics
from sklearn.datasets import make_blobs
```

2.5.2 Create sample data

We'll use `make_blobs` to create synthetic 2D data points.

```
# Create dataset with 4 centers (clusters)
X, y_true = make_blobs(n_samples=300, centers=4,
                       cluster_std=0.50, random_state=0)

# Visualize the raw data
plt.scatter(X[:, 0], X[:, 1], s=50)
plt.title("Original Data (Unclustered)")
plt.show()
```



2.5.3 Apply DBSCAN

```
db = DBSCAN(eps=0.3, min_samples=10).fit(X)

# Create a boolean mask initialized to False for all points
# This mask will later mark which points are core samples identified by
# DBSCAN
core_samples_mask = np.zeros_like(db.labels_, dtype=bool)

# Mark the core sample indices obtained from DBSCAN as True in the mask
core_samples_mask[db.core_sample_indices_] = True

# Extract the cluster labels assigned by DBSCAN to each point in the
# dataset
# Each point gets:
#   - A cluster label (0, 1, 2, ...)
#   - Or label = -1 for noise/outliers
labels = db.labels_
```

```

# Count the number of clusters found by DBSCAN
# DBSCAN may assign some points to noise (-1), so we subtract 1 if noise
exists
n_clusters_ = len(set(labels)) - (1 if -1 in labels else 0)
print(f'Estimated number of clusters: {n_clusters_}')

# Get the set of unique labels produced by DBSCAN
# This includes cluster IDs and possibly -1 for noise
unique_labels = set(labels)
print(unique_labels)

```

```

Estimated number of clusters: 4
{np.int64(0), np.int64(1), np.int64(2), np.int64(3), np.int64(-1)}

```

2.5.4 Visualize the clustered data

```

# Generate one color per cluster (excluding noise)
colors = plt.cm.Spectral(np.linspace(0, 1, len(unique_labels)))

plt.figure(figsize=(7, 6))

for k, col in zip(unique_labels, colors):

    # Noise is colored black
    if k == -1:
        col = 'k'

    class_member_mask = (labels == k)

    # Plot core points
    xy = X[class_member_mask & core_samples_mask]
    plt.plot(
        xy[:, 0], xy[:, 1], 'o',
        markerfacecolor=col,
        markeredgecolor='k',
        markersize=8
    )

```

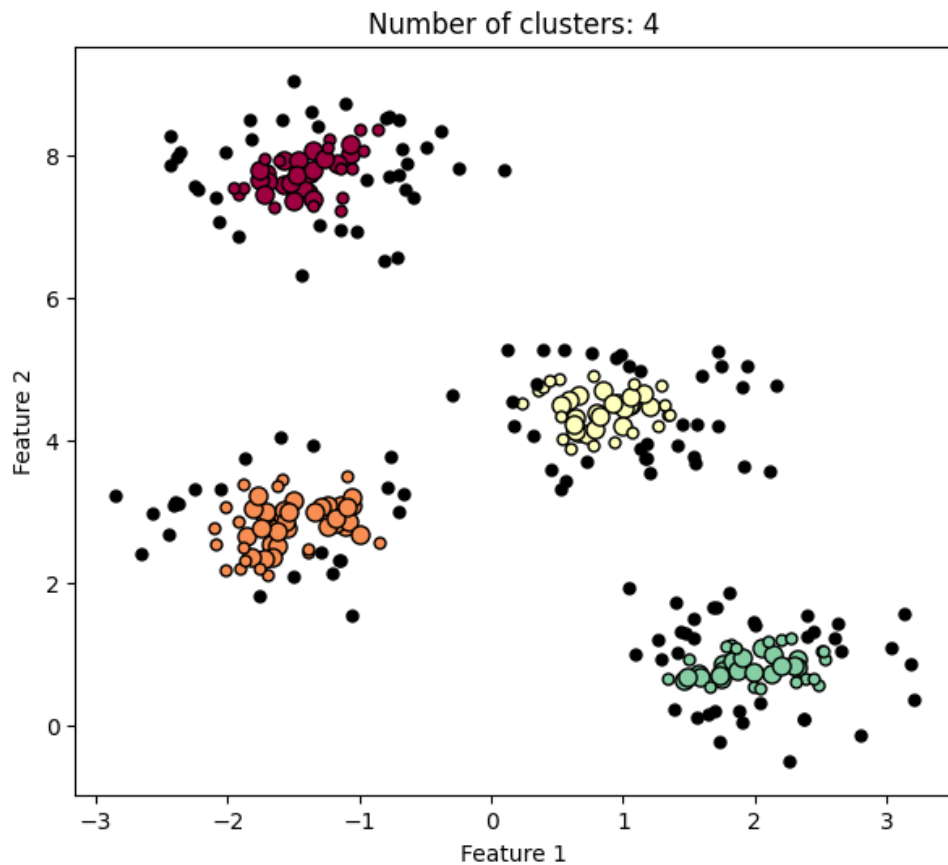


```

# Plot border points
xy = X[class_member_mask & ~core_samples_mask]
plt.plot(
    xy[:, 0], xy[:, 1], 'o',
    markerfacecolor=col,
    markeredgecolor='k',
    markersize=5
)

plt.title(f"Number of clusters: {n_clusters}")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()

```



2.1.5 Evaluation Metrics

```

sc = metrics.silhouette_score(X, labels)
print("Silhouette Coefficient:%0.2f" % sc)
ari = metrics.adjusted_rand_score(y_true, labels)
print("Adjusted Rand Index: %0.2f" % ari)

```

```

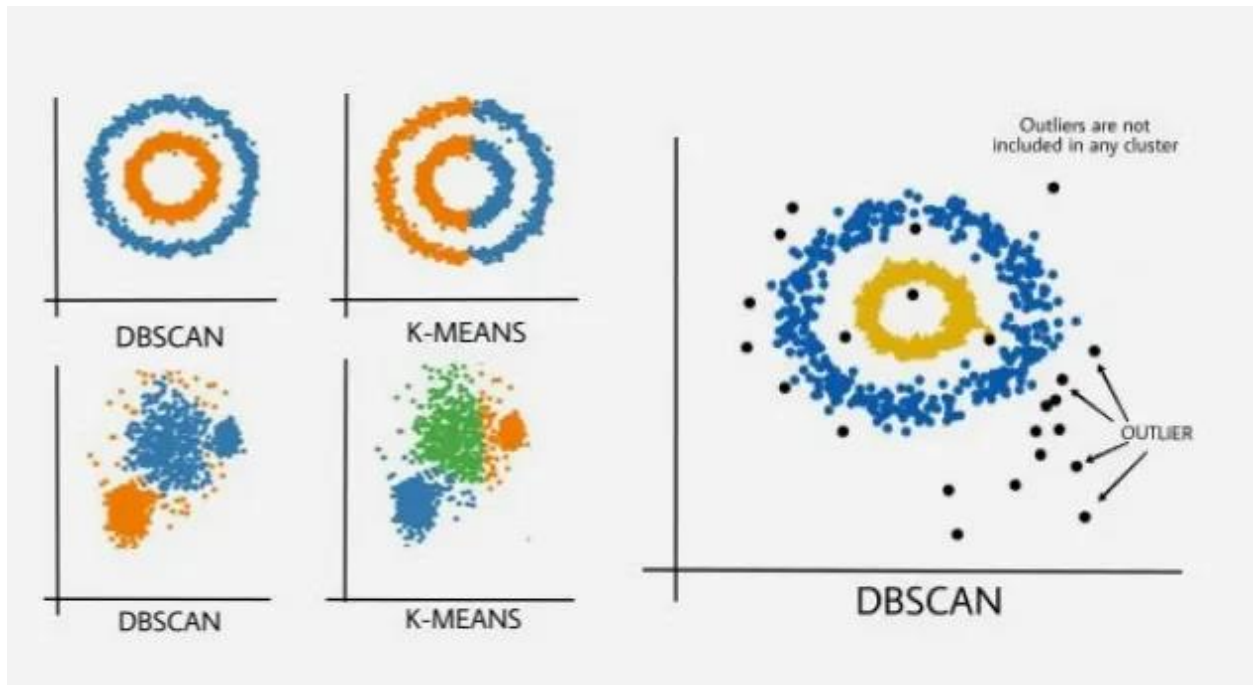
Silhouette Coefficient:0.13
Adjusted Rand Index: 0.31

```

2.6 When Should We Use DBSCAN Over K-Means Clustering?

DBSCAN and K-Means are both clustering algorithms that group together data that have the same characteristic. However they work on different principles and are suitable for different types of data. We prefer to use DBSCAN when the data is not spherical in shape or the number of classes is not known beforehand.

Aspect	DBSCAN	K-Means
Number of clusters	Does not require specifying the number of clusters in advance.	Requires specifying the number of clusters (k) before training.
Cluster shape	Can form clusters of arbitrary shape.	Forms spherical or convex clusters.
Handling noise & outliers	Works well with datasets containing noise and outliers; outliers are labeled as noise (-1).	Performs poorly with outliers; outliers can skew the cluster centers significantly.
Parameters required	Requires two parameters: eps (neighborhood radius) and min_samples.	Requires one main parameter: number of clusters k.



As it can identify clusters of arbitrary shapes and effectively handle noise. K-Means on the other hand is better suited for data with well-defined, spherical clusters and is less effective with noise or complex cluster structures.

Reference: <https://www.geeksforgeeks.org/machine-learning/dbscan-clustering-in-ml-density-based-clustering/>

Exercises

Task 1:

1. Understand and implement the code provided in the attached lab manual to implement DBSCAN using built in libraries.

Task 2:

1. Search and download any unsupervised dataset suitable for DBSCAN (e.g., from UCI Machine Learning Repository, Kaggle, or sklearn datasets).
2. Explore the dataset: check dimensions, data types, missing values, and basic statistics. Perform any necessary data cleaning operations.
3. Perform DBSCAN on the dataset using builtin libraries.
4. Evaluate the clustering results using appropriate evaluation metrics.